- **OGR_RESTO : 11. Contrôles avancés (5 programmes annexes)**
  - **Cette étape finale effectue 5 vérifications croisées pour détecter des cas particuliers de rapprochement qui n'ont pas été détectés par les étapes précédentes. Chaque programme traite un cas spécifique.**
    - 11.1 — Vérification 1 : verification_noms.sas (Noms similaires)
      - Problème traité
        - Des fiches sont liées par une clé adresse large (ex: cle_adr_complete, cle_comp_ss_num) mais ont des noms légèrement différents. Sans vérification, elles ne seraient pas validées automatiquement.
      - Logique détaillée
        - Étape 1 : Sélection des codens concernés
          - Critères :
            - com_vali="" : pas de commentaire de validation (pas de fermeture détectée)
            - codens commence par : ADR_COMPLETE_, COMP_SS_NUM_, PCOMPL_SS_NUM_, ADR_PCOMP_, PCOMPL2_, PCOMPL2_SS_NUM_, PCOMPL_
          - → Codens basés sur clés larges (sans dénomination ou sans détails d'adresse)
        - Étape 2 : Transposition des dénominations
          - Résultat : une ligne par codens avec les dénominations en colonnes
            - Avant :
              - 75_ADR_COMPLETE_001 — LOASIS — denomination_f
              - 75_ADR_COMPLETE_001 — LOASISKEBAB — denomination_f
              - 75_ADR_COMPLETE_001 — OASISKEBAB — denomination_f
            - Après :
              - 75_ADR_COMPLETE_001 — LOASIS — COL1
              - LOASISKEBAB — COL2
              - OASISKEBAB — COL3
        - Étape 3 : Calcul de distance de Levenshtein
          - → Un nom est inclus dans l'autre (ex: "OASIS" + 5 caractères = "OASIS KEBAB")
          - Exemple :
            - COL1 = "LOASIS" (longueur 6)
            - COL2 = "LOASISKEBAB" (longueur 11)
            - complev("LOASIS", "LOASISKEBAB") = 5 — complev() : fonction SAS calculant la distance de Levenshtein (nombre de modifications nécessaires pour passer d'une chaîne à l'autre).
            - Test : 6 + 5 = 11 ✓
            - → ok1=1, ok2=1, ok_1_2=1
        - Étape 4 : Détection des groupes et split si nécessaire
          - Si un codens regroupe 4 fiches dont 2 paires distinctes, on splitte le codens en 2 :
          - Exemple :
            - Codens 75_ADR_COMPLETE_001 :
              - Fiche 1 : "PIZZERIA NAPOLI"
              - Fiche 2 : "PIZZERIA NAPOLI ROMA" (ok_1_2=1, similaires)
              - Fiche 3 : "SUSHI KING"
              - Fiche 4 : "SUSHI KING ROYAL" (ok_3_4=1, similaires)

              Détection : ok_1_2=1 et ok_3_4=1 mais ok_1_3=, ok_1_4=, ok_2_3=, ok_2_4=. → 2 groupes distincts ! Action : Split en 2 codens
              - SPLIT12_75_ADR_COMPLETE_001 (fiches 1 et 2)
              - SPLIT34_75_ADR_COMPLETE_001 (fiches 3 et 4)

              Le code gère jusqu'à 5 fiches avec des patterns complexes (doublets, triplets, etc.).
        - Étape 5 : Attribution du nouveau codens et couleur
          - Logique :
            - Si la dénomination de la fiche correspond à une dénomination "OK" (similaire détectée) :
              - Nouveau codens attribué (ou conservé ou splité)
              - couleur_fin=1 → Validation automatique
            - Sinon :
              - Codens vidé (fiche non similaire)
      - Exemples :
        - "L'OASIS" et "L'OASIS KEBAB" (même restaurant, nom étendu)
        - "PIZZA NAPOLI" et "PIZZERIA NAPOLI" (variante orthographique)
        - "CAFE DE PARIS" et "CAFE PARIS" (préposition manquante)
      - Solution
        - Calcul de distance de Levenshtein
    - 11.2 — Vérification 2 : verification_codens_multi_clenum.sas (Codens multi-clenum)
      - Problème traité
        - Un codens regroupe plusieurs fiches N-1 (plusieurs clenum différents). Cela arrive typiquement dans les centres commerciaux où plusieurs restaurants partagent la même adresse.
      - Logique détaillée
        - Étape 1 : Identification des codens problématiques
          - Critères :
            - nb_clenum>1 : au moins 2 fiches N-1 différentes (clenum différents)
            - nb_fiches<6 : moins de 6 fiches totales (au-delà, trop complexe, on vide le codens)
            - substr(codens,1,5) ne "SPLIT" : exclut les codens déjà splittés à l'étape 1
        - Étape 2 : Analyse des similarités de noms
          - Même logique que verification_noms.sas
            - 1 - Transposition des dénominations
            - 2 - Calcul distances de Levenshtein
            - 3 - Détection des groupes similaires
            - 4 – Split des codens en sous-groupes
          - Résultat :
            - Avant : Codens 75_ADR_COMPLETE_042 (4 fiches, 2 clenum)
            - Après split : Codens SPLIT12_75_ADR_COMPLETE_042 (fiches 1 et 3, "PIZZERIA NAPOLI") / Codens SPLIT34_75_ADR_COMPLETE_042 (fiches 2 et 4, "SUSHI KING")
          - Cas particulier : plus de 5 fiches
            - → Si un codens a plus de 5 fiches ET plusieurs clenum, il est vidé complètement (trop complexe à analyser automatiquement).
    - 11.3 — Vérification 3 : verification_codens_n1.sas (Cohérence codens_n1)
      - Problème traité
        - Des fiches liées (même codens) dont au moins une est issue de N-1 (clenum rempli), et toutes ont le même codens_n1.
      - Logique détaillée
        - Étape 1 : Vidage des codens_n1 uniques
          - → Si un codens_n1 n'apparaît qu'une fois, on le vide (pas d'intérêt pour le rapprochement).
        - Étape 2 : Identification des fiches cohérentes
          - Critères :
            - Fiches non validées auto (couleur_fin ne 1)
            - Fiches liées (même codens)
            - Au moins une fiche N-1 (clenum ne "")
            - Toutes les fiches liées ont le même codens_n1
        - Étape 3 : Création nouveau codens et validation auto
          - Résultat :
            - Avant :
              - Fiche 1 : codens="75_PCOMPL_042", codens_n1="75_COMPLETE_00123", couleur_fin=2
              - Fiche 2 : codens="75_PCOMPL_042", codens_n1="75_COMPLETE_00123", couleur_fin=2
            - Après :
              - Fiche 1 : codens="NEW_75_COMPLETE_00123", couleur_fin=1 ✅
              - Fiche 2 : codens="NEW_75_COMPLETE_00123", couleur_fin=1 ✅
        - Étape 4 : Cas sans codens
          - Le programme traite aussi les fiches sans codens mais avec même codens_n1 :
            - → Si plusieurs fiches sans codens partagent le même codens_n1, on leur crée un nouveau codens et on valide auto.
      - Situation :
        - Codens 75_PCOMPL_042 :
          - Fiche 1 (N-1) : clenum=1001, codens_n1="75_COMPLETE_00123", couleur_fin=2
          - Fiche 2 (N) : clenum="", codens_n1="75_COMPLETE_00123", couleur_fin=2
          - Fiche 3 (N) : clenum="", codens_n1="75_COMPLETE_00123", couleur_fin=2
        - → Les 3 fiches ont été liées à la même fiche N-1 (même codens_n1)
          - → C'est probablement le même restaurant, on peut valider automatiquement !
    - 11.4 — Vérification 4 : verification_siren.sas (SIREN identiques)
      - Problème traité
        - Des fiches sans codens, non validées auto, mais avec le même SIREN qu'une fiche déjà liée (avec codens).
      - Logique détaillée
        - Étape 1 : Sélection des fiches orphelines avec SIREN
        - Étape 2 : Rapprochement par SIREN + codinsee
          - Double condition :
            - Même SIREN
            - Même code INSEE (même commune)

            + Garantit que c'est le même établissement (un SIREN peut avoir plusieurs établissements dans différentes communes).
        - Étape 3 : Attribution du codens
          - → Si la fiche "autres" n'avait pas de codens, on en crée un nouveau.
          - → Les fiches orphelines récupèrent ce codens.
        - Étape 4 : Gestion des SIREN multiples dans un même codens
          - Problème détecté : un codens regroupe des fiches avec plusieurs SIREN différents.
          - Cause : elles avaient été liées par une clé adresse seule (cle_adr_complete).
          - Solution : on crée un codens par SIREN.
            - Résultat :
              - Avant :
                - Fiche 1 : codens="75_ADR_COMPLETE_042", siren="111111111"
                - Fiche 2 : codens="75_ADR_COMPLETE_042", siren="222222222"
                - Fiche 3 : codens="", siren="111111111"
              - Après :
                - Fiche 1 : codens="NEW_SIREN_75_111111111"
                - Fiche 2 : codens="NEW_SIREN_75_222222222"
                - Fiche 3 : codens="NEW_SIREN_75_111111111" ← Rattachée à Fiche 1
      - Exemple :
        - Fiche 1 : codens="75_COMPLETE_042", siren="123456789", codinsee="75011"
        - Fiche 2 : codens="", siren="123456789", codinsee="75011", couleur_fin=4

        + Fiche 2 devrait être rattachée à Fiche 1 (même entreprise, même commune).
    - 11.6 — Vérification 5 : verification_denom_codpost.sas (Dénomination + Code postal)
      - Problème traité
        - Des restaurants avec la même dénomination ET le même code postal mais sans codens ou dans des codens différents.
        - Cas typique : chaînes de restaurants
          - "PAUL" présent dans plusieurs arrondissements de Paris
          - "BRIOCHE DOREE" présent dans plusieurs rues du même arrondissement
          - "MCDONALD'S" avec plusieurs établissements
      - Logique détaillée
        - Étape 1 : Identification des dénominations répétées
          - Critères :
            - nb_fiches>1 : au moins 2 fiches avec même dénomination + code postal
            - nb_clenum<2 : au maximum 1 fiche N-1 (évite les cas déjà traités)
        - Étape 2 : Analyse des noms de voies
          - Même logique que verification_noms.sas appliquée aux noms de voies :
            - → On transpose les noms de voies (pas les dénominations) pour voir si elles sont similaires.
          - Logique :
            - Si les noms de voies sont similaires (distance Levenshtein) → probablement même restaurant, adresses légèrement différentes
            - Sinon → probablement plusieurs établissements de la même chaîne
        - Étape 3 : Création de nouveaux codens
      - Résultat :
        - Avant :
          - Fiche 1 : "PAUL", codpost="75001", nomvoie="RIVOLI", codens=""
          - Fiche 2 : "PAUL", codpost="75001", nomvoie="LOUVRE", codens=""
        - Après (si voies similaires) :
          - Fiche 1 : codens="NEW_DENOM_75_001"
          - Fiche 2 : codens="NEW_DENOM_75_001"
        - Après (si voies différentes) :
          - Fiche 1 : codens="NEW_DENOM_75_001"
          - Fiche 2 : codens="NEW_DENOM_75_002"

Presented with xmind